

课程笔记

Codex 中的 Skills 与 Tools 加载: Context Engineering、Agent Runtime 与 Agent Loop

五道口纳什 & Codex

2026 年 6 月 25 日

modern AI Agent Skills 与 Tools Agent Runtime Agent Loop

视频作者/频道: 五道口纳什

发布日期: 2026 年 3 月 13 日

视频时长: 10 分 51 秒

视频链接: <https://www.bilibili.com/video/BV1bSwpzNEWE/>

目录

1 现代 AI Agent 的问题背景	2
1.1 从系列语境进入 Context Engineering	2
1.2 为什么 Context Engineering 成为主线	3
1.3 Tools、MCP 与 Skills 的问题引出	3
1.4 本章小结	3
2 现代 Agent 的三个能力层	4
2.1 第一层：Foundation Model 的原始推理能力	4
2.2 第二层：Long-Horizon Agentic 多轮执行	4
2.3 第三层：Runtime 注入、工具与调度编排	4
2.4 本章小结	5
3 Agent Runtime 与上下文工程	5
3.1 Runtime 是 Agent 的执行环境	5
3.2 Message List：模型真正收到的任务包裹	5
3.3 Context Engineering 的精确定义	6
3.4 从 Runtime 走向 Agent Loop	7
3.5 本章小结	7
4 能力必须暴露在 Context 中	8
4.1 从“模型能力”转到“可见上下文”	8
4.2 一个简化的上下文公式	8
4.3 为什么这是 Context Engineering 的入口	8
4.4 本章小结	9
5 Skills 与 Tools 的位置差异	9
5.1 同属能力系统，进入 Context 的位置不同	9
5.2 Tools/MCP 通过 API 的 tools 字段暴露	9
5.3 为什么 Skills 不会自动降低 Tools 的 Context 成本	10
5.4 Skills-vs-Tools 图示	10
5.5 本章小结	10
6 Agent Loop：从输入到动作再到持久化	11
6.1 Agent Runtime 是舞台，Agent Loop 是流程	11
6.2 从 Intake 到 Persistence 的控制流	12
6.3 Function Call 把 Loop 接到真实世界	12
6.4 转向 Codex Tools Schema	13
6.5 本章小结	13
7 工具枚举、MCP 动态加载与落盘观察	13
7.1 为什么要把 Tools Schema 单独拿出来看	13
7.2 Codex 观察到的五类工具模式	14
7.3 MCP 是动态工具来源	15

7.4	本地落盘：从黑箱请求到可检查证据	15
7.5	本章小结	16
8	Response API 请求体结构	16
8.1	从日志里看到 response.create	16
8.2	四个主轴：instructions、input、previous response、tools	17
8.3	一个简化公式：Context 由消息、工具和运行时链接组成	18
8.4	容易误读的地方	18
8.5	本章小结	19
9	Codex 默认工具与多 Agent 能力	19
9.1	没有配置 MCP 时的默认函数清单	19
9.2	单代理工具：执行、编辑、检索、协商	20
9.3	多 Agent 工具：把长任务拆成可汇总的并行工作	20
9.4	从工具清单回到 Context Engineering	21
9.5	本章小结	21
10	总结与延伸	22
10.1	全片核心：能力必须进入模型可见请求	22
10.2	Skills、Tools、MCP 的统一视角	22
10.3	用一个 Agent Loop 公式串起来	23
10.4	工程启示：从聊天界面到可编程运行时	23
10.5	继续追问：版本、权限、日志和安全	23
10.6	复盘操作清单	24

1 现代 AI Agent 的问题背景

1.1 从系列语境进入 Context Engineering

本讲开场真正有教学价值的部分，是把 **Modern AI Agent** 放进 **Context Engineering** 这个更大的问题框架里。演讲者提到的代表包括 Codex、Claude Code、Gemini CLI 与 OpenClaw 一类系统。它们的共同点集中在一套围绕大模型推理建立起来的运行机制：用户提出任务，运行时组织上下文，模型在上下文里推理，工具被调用，结果再回到上下文中。

这也是本视频区别于普通产品介绍的地方。它关心的是：现代 Agent 的能力如何被放进模型可见的请求里？如果能力只存在于本地文件、外部工具、MCP server 或某个文档目录中，却没有被运行时暴露给模型，那么模型在本次推理中就无法把它当成可用能力处理。这个问题贯穿后续关于 **Skills**、**Tools**、**MCP**、**Agent Runtime** 与 **Agent Loop** 的讨论。

术语定位：Modern AI Agent

本讲里的 **Modern AI Agent** 指一种面向任务执行的现代代理系统。它通常结合更强的 foundation model、更长的 *agentic* 多轮执行能力、运行时注入的上下文、工具调用能力，以及调度编排机制。读者可以把它理解成“带执行环境的大模型工作系统”：模型负责推理，运行时负责让推理发生在足够完整、足够可操作的上下文中。

1.2 为什么 Context Engineering 成为主线

早期使用大模型时，很多讨论集中在 prompt 本身：怎样写指令、怎样补充示例、怎样让模型遵循格式。现代 Agent 的复杂度更高，因为 prompt 只是模型可见上下文的一部分。实际请求里还可能包含系统指令、developer 消息、user 消息、历史对话、函数输出、技能摘要、记忆材料，以及 API 层面的 tool schema。

因此，**Context Engineering** 可以理解为对这些材料的工程化组织。它研究的对象包括三类问题：

- 哪些材料应该进入模型可见请求，例如 `agents.md`、`skills.md`、`memory.md`、用户 query、历史结果。
- 这些材料以什么形式进入请求，例如 message content、function output、top-level instructions、API 的 tools 字段。
- 这些材料在什么时机进入请求，例如启动时预注入、用户首次提问后匹配 Skill、工具调用后回填 function output。

一个直观类比是“给模型打包任务包裹”。模型只打开这次请求里的包裹；包裹外面确实可能有文件、网页、数据库和工具，但运行时没有把它们放进包裹，模型就只能从已经放入的材料出发。

本讲的核心问题

现代 Agent 的关键问题可以压缩为一句话：运行时如何决定什么进入模型可见上下文、进入哪个请求表面、以及在 Agent 执行过程中何时更新上下文。

1.3 Tools、MCP 与 Skills 的问题引出

视频在开头就提出一个重要判断：**Tools/MCP** 与 **Skills** 在技术结构上是正交、独立、解耦的。这里的“正交”借用数学直觉来说明工程系统里的层次独立：一个机制解决“文档化能力如何进入消息内容”，另一个机制解决“可调用工具如何通过 API schema 暴露给模型”。

这一区分很关键。Skill 往往像一本说明书，告诉模型某类任务的流程、约束和可用资源；Tool schema 则像一张可执行接口清单，告诉模型某个函数叫什么、参数是什么、返回什么。说明书可以提到某个工具的使用场景，但接口清单仍需要由 API 层注册。后续章节会专门展开这个差异。

名称与字幕误差

本视频字幕里有若干 AI 转写误差，例如 Tools 被写成类似“tooth”，Codex 被写成类似“Cortex/Pocket X”，Claude Code 被写成类似“cloud code”。本笔记按技术语境校正这些术语；视频总览画面显示该处产品名为 OpenClaw。

1.4 本章小结

本章建立了全片的入口：现代 AI Agent 的能力来源需要从上下文工程来理解。演讲者讨论的焦点，是 Codex、Claude Code、Gemini CLI 等系统背后共同的运行逻辑：运行时把消息、技能、工具、记忆与历史状态组织成模型可见请求，模型只能基于这些材料推理和行动。接下来的问题是，这些系统究竟由哪几层能力共同支撑。

2 现代 Agent 的三个能力层

2.1 第一层：Foundation Model 的原始推理能力

演讲者首先指出，现代 AI Agent 的底层变化来自更强的 **foundation model**。更强模型意味着更好的语言理解、更稳定的代码生成、更长的推理链条、更强的指令遵循能力。没有这一层，后续的运行时、工具调用、长程任务拆解都会失去基本支撑。

这里要谨慎处理字幕里的模型版本号。字幕中出现了类似 GPT 与 Opus 系列的说法，但自动字幕可能误识别具体版本。成稿应保留“更强基础模型”这个结构性判断，避免把未核验的具体版本号写成事实清单。

基础模型像发动机

可以把 foundation model 类比为发动机：发动机越强，车辆越有潜力完成复杂路况。但车辆能否跑长途，还取决于导航、油路、刹车、仪表盘、维护系统。现代 Agent 也是这样，模型能力是基础，运行时与工具编排决定这份能力能否稳定变成长期任务执行。

2.2 第二层：Long-Horizon Agentic 多轮执行

第二层能力是更长程的 **agentic** 多轮执行。视频里说，用户发送一个 query 之后，Agent 会经历较长的多轮交互，最终写出完整代码或满足用户要求。这里的重点是“长程”：系统需要在多步执行中保留目标、分解任务、调用工具、观察结果、修正计划，并持续把中间结果带回后续推理。

普通聊天模型更像一次性问答；现代 Agent 更像持续推进的任务执行者。一个代码任务可能先读文件，再制定计划，再修改代码，再跑测试，再根据失败输出回到源码。这种循环需要上下文承载任务状态，也需要工具把外部世界的反馈带回来。

长程执行的最低要求

Long-horizon agentic execution 至少需要三件事：任务目标能被保留，中间观察能回到上下文，下一步动作能基于前一步结果调整。缺少其中任何一项，系统都会退化成短回合问答。

2.3 第三层：Runtime 注入、工具与调度编排

第三层能力来自 **Agent Runtime**。字幕中提到，运行时会准备大量 prompt injection，例如 `agents.md`、`skills.md`、`memory.md`；以 Codex 为例，用户第一次发起请求时，请求里已经可能包含 `system instruction`、`developer message` 和 `user message`。其中 `user message` 还可能包含 `agents.md` 以及可用 Skills 的摘要。

同时，现代 Agent 通常内置多类工具能力。视频里举到的例子包括 **Spawn Agents**、**Update Plan**、**Shell Commands**、**Web Search**、**Web Fetch**，以及浏览器自动化、App 交互、MCP servers 等。工具让模型能够读取环境、执行命令、搜索资料、更新计划、调度子代理；调度编排则负责让这些动作在一个 Agent Loop 中有顺序地发生。

为了把三层能力压缩成一个教学模型，可以写成：

$$A_{\text{agent}} \approx F_{\text{model}} + H_{\text{horizon}} + R_{\text{runtime}}$$

- A_{agent} ：一个现代 Agent 在任务中的实际有效能力。

- F_{model} : foundation model 的原始理解、生成、推理能力。
- H_{horizon} : 长程多轮任务推进能力, 包括计划、观察、修正与持续执行。
- R_{runtime} : 运行时提供的上下文注入、工具 schema、Skills、memory、调度和本地/远端 API 调用环境。

这个公式只是教学抽象。它的价值在于提醒读者: 现代 Agent 的有效能力来自三层共同作用。只看模型参数, 会漏掉运行时对上下文和工具的组织; 只看工具清单, 会漏掉模型是否能在长程任务中稳定使用这些工具。

常见误解: 把 Agent 能力压成模型能力

如果把现代 Agent 的能力全部归因于 foundation model, 就会误判系统设计的关键。模型再强, 也需要 runtime 把系统指令、developer 约束、user query、Skill 摘要、memory 和 tools 暴露到本次请求里。没有这些可见材料, 模型无法凭空知道本地有哪些能力可用。

2.4 本章小结

现代 Agent 可以从三层能力理解: 第一层是 foundation model, 提供推理与生成基础; 第二层是 long-horizon agentic execution, 让系统能在多轮任务中保持目标并持续调整; 第三层是 Agent Runtime 与工具编排, 把文件、消息、技能、记忆、工具和调度流程组织起来。第 3 节将进一步说明 Runtime 为什么会成为 Context Engineering 的核心执行位置。

3 Agent Runtime 与上下文工程

3.1 Runtime 是 Agent 的执行环境

Agent Runtime 可以理解为现代 Agent 的执行环境。演讲者把它描述为维护和管理 **Agent Loop** 的运行时系统: 它负责准备 prompt injection、Skills 文档、项目规则、技能摘要、memory 片段、内置 Tools, 以及其他与工作区相关的部分。模型推理面对的是 runtime 为本次 API 调用组装出来的请求, 而非整个电脑和互联网。

这里的“运行时”很像舞台系统。演员是模型, 剧本片段是消息内容, 灯光和道具是工具与环境状态, 后台调度决定什么时候把什么材料推到台前。观众看到的是台上的表演; 工程师需要关心后台如何保证每个关键材料按时出现。

Agent Runtime 的核心职责

Agent Runtime 的核心职责, 是把用户任务、系统约束、开发者约束、技能材料、记忆、工具定义和历史输出组织成模型可见请求, 并在 Agent Loop 中持续更新这些材料。

3.2 Message List: 模型真正收到的任务包裹

视频中反复强调, 现代 Agent 最后仍然要调用本地或远端 API。对做过 API 调用的读者来说, 关键对象就是 **Message List**: 一组按顺序排列的消息。它可以包含 system instruction、developer message、user message、assistant 回复、function output 等。Codex 的例子里, 用户第一次发起请求时, runtime 已经提前植入多条消息, 其中 user message 里还会带上 agents.md 和 Skill 摘要。

把 API 请求想成“寄给模型的包裹”会更直观。包裹里有任务说明、约束、背景资料、可调用工具清单和历史结果；模型打开包裹后，只能基于这些材料做下一步推理。文件系统、网页、MCP server、浏览器自动化能力都可能存在于外部世界，但只有经过 runtime 暴露后，它们才会成为本次推理的可用上下文。

Message content 与 tool schema 是两个请求表面

Message content 是写进消息里的文字材料，例如 `agents.md`、Skill 摘要、用户问题和 function output。Tool schema 是 API 层面的结构化工具定义，通常放在 `tools` 字段里。两者都影响模型可见上下文，但进入请求的位置不同。后续关于 Skills、Tools 与 MCP 的章节会围绕这个区别展开。

3.3 Context Engineering 的精确定义

在本讲语境下，**Context Engineering** 指一项运行时工程，范围远大于“写好提示词”：决定消息列表里有什么、工具 schema 里有什么、历史状态如何连接、函数输出如何回填、哪些材料留在本地、哪些材料进入本次请求。演讲者说“所有 Modern AI Agent 都是在做 context engineering”，指的就是这套围绕模型可见请求的工程活动。

为了便于后续章节复用，可以把模型可见上下文写成一个简化表达：

$$C_{\text{visible}} \approx M \cup T \cup R$$

- C ：模型在一次推理中可见、可依据的整体上下文。
- M ：Message List，包括 system/developer/user 消息、assistant 消息和 function output。
- T ：API 层面的 Tools Schema 集合，例如内置函数、Web Search、Shell Command、MCP 暴露的函数。
- R ：Runtime 管理的连接与状态，例如 previous response linkage、任务队列、本地持久化记录等。

这个表达是教学模型，用来辅助区分三类材料：写进消息的内容、注册在工具字段里的可调用能力、由运行时维护的状态连接。后续讨论 Skills 与 Tools 的差异时，这个区分会直接派上用场。

Context Engineering of Codex/CC/OpenClaw

- Codex/CC/Gemini-CLI/OpenClaw 都是 modern AI Agent
 - 更强的基础模型，更 (long-horizon) agentic turn
 - agent runtime: 提示词注入，skills / tools: 浏览器自动化 / App 交互
 - 更合适的调度编排系统 (agent loop)
- agent runtime => agent loop => api messages => context
 - 在做 context engineering
- 不管是 skills 还是 mcp 或者任何预设的 tools 不暴露出现在 context (messages) 里，model 是不可能感知、调用的；
 - 当然还包括 memory；
- skills 跟 mcp (技术上) 正交、独立、解耦
 - 不存在说出现了 skills 之后，简化了 mcp/tools 所占的 context，除非显式设计
 - skills 摘要出现在 message content (inject user message)
 - together with [AGNETS.md](#)
 - [skills.md](#) (正文)通过 cat 命令加载到 context 里
 - mcp 则是 tools schema
 - skills 中可以提及具体的 tool：即可以说明什么时候用什么样的 tool

图 1: 视频中的上下文工程总览：Modern AI Agent、Agent Runtime、Message/API Context 与 Skills/Tools/MCP 的位置关系。该图是后续章节的结构索引。¹

3.4 从 Runtime 走向 Agent Loop

Runtime 与 Agent Loop 是互补概念。Runtime 是执行环境和上下文组织者；Agent Loop 是消息进入系统之后反复发生的控制流程。字幕中后续会把 loop 描述为：intake 接收消息，可能进入队列，准备 prompt injection，模型推理，需要时发起 function call，执行工具，生成 streaming response，再持久化到本地。

本章先把重点放在 Runtime，因为没有 Runtime 的上下文组织，Agent Loop 只是一串抽象步骤。真正使 loop 可执行的是 runtime 每一步都能把新的观察、工具结果和状态更新回填到后续请求中。

上下文缺失带来的硬边界

模型无法调用自己看不见的能力。Skill 文件、MCP server、浏览器自动化和本地工具即使已经存在于系统里，只要 runtime 没有把相应说明或 schema 暴露给模型，它们在本次推理中就等同于不可用资源。

3.5 本章小结

Agent Runtime 是现代 Agent 做 Context Engineering 的关键位置。它把启动文件、角色消息、Skill 摘要、memory、工具 schema、function output 与历史状态组织成模型可见请求；Message List 则是模型实际收到的任务包裹。理解这一点后，读者就能自然进入下一组问题：Skills、Tools 与 MCP 分别通过什么请求表面进入上下文，以及为什么它们在技术上保持正交。

¹ 视频画面时间区间：00:03:40-00:04:00。若为裁剪图，时间区间仍对应原视频画面。

4 能力必须暴露在 Context 中

4.1 从“模型能力”转到“可见上下文”

这一段的核心判断很硬：现代 AI Agent 的能力并不会因为系统外部存在某个 Skill、MCP server、Tool 或 memory 文件就自动生效。模型真正能利用的材料，必须进入 **model-visible request context**，也就是一次推理请求中模型可见的消息内容、工具定义、函数输出和运行时注入的信息。

用直觉类比，模型像收到一个包裹的审阅者。仓库里也许有很多工具、手册和历史记录；审阅者能依据的只有包裹里实际装进去的东西。若某个 Skill 只躺在磁盘上，若某个 MCP server 只存在于配置之外，若某段 memory 没有被运行时放入请求，模型就无法感知它们的存在，更谈不上主动调用。

Context Exposure Rule

模型只能使用已经暴露到 **model-visible request context** 的能力。这里的“暴露”可以发生在 message content 中，也可以发生在 API 的 **tools** 字段中，还可以表现为一次 Tool 调用之后返回的 **FunctionOutput** 消息。缺席于这些表面的能力，在本次推理里等价于不存在。

4.2 一个简化的上下文公式

为了把这一点说清楚，可以把一次推理看到的上下文写成一个教学用公式。该公式属于教学压缩，视频没有把它作为正式数学推导：

$$C_{\text{visible}} \approx M \cup T \cup R$$

- C_{visible} ：模型本次推理可见的整体上下文。
- M ：message list，包括 system/developer/user/assistant/function output 等消息内容。
- T ：通过 API **tools** 字段暴露的工具 schema 集合，例如 built-in Tools、MCP functions、custom functions。
- R ：Agent Runtime 维护并注入的运行状态或链接信息，例如前序响应关联、已选择的文件内容、memory 摘要、会话材料。

这个公式的意义在于提醒读者：上下文工程（**Context Engineering**）的对象是整包请求，范围远远超过一句 prompt。它关心什么材料进入 M ，什么工具定义进入 T ，什么运行时状态进入 R ，以及这些材料在什么时候进入。

Message List 与 Tools Field 的两条通道

Message List 像讲义正文，适合放说明、约束、示例、历史对话和函数输出。**Tools Field** 像工具目录，适合放可调用函数的名称、参数结构和调用规则。二者都会影响模型行为，但它们进入请求的位置和结构不同。

4.3 为什么这是 Context Engineering 的入口

讲者把这个规则放在 Skills 与 Tools 讨论之前，原因很直接：若读者先把 Agent 理解成“一个更聪明的模型”，就会漏掉最关键的工程层。现代 AI Agent 的工程重点，在于 Agent Runtime 如何

维护上下文：哪些启动文件被读入，哪些 skill summary 被注入，哪些 Tools Schema 被注册，哪些 memory 进入请求，哪些函数调用结果重新回到 message list。

这也解释了为什么同一个基础模型，在不同 Agent Runtime 里表现差异巨大。模型参数可能相同，实际可见的上下文包裹却不同。一个 runtime 提供了 shell、web search、view image、spawn agents、update plan 等工具 schema，并在消息里注入项目规则；另一个 runtime 只发送用户原始问题。两者的可行动范围自然不同。

常见误解：文件存在不等于模型可见

把 AGENTS.md、SKILL.md 或 memory 文件放在项目里，只表示运行时可能读取它们。若运行时没有把相关内容放进 message list，或没有通过 Tool 调用把完整文件读回上下文，模型无法凭空知道这些文件的细节。

4.4 本章小结

本章建立了后续三节的地基：现代 AI Agent 的能力来自模型与上下文暴露机制的组合。Skill、Tool、MCP、memory 的共同前提是可见性；它们只有进入 model-visible request context，才会成为模型推理和行动的一部分。Context Engineering 的第一问因此是：这项能力到底从哪里进入请求、以什么结构进入、在什么时刻进入。

5 Skills 与 Tools 的位置差异

5.1 同属能力系统，进入 Context 的位置不同

讲者接着给出本讲最容易被误解的一点：**Skills** 与 **Tools/MCP** 是正交、独立、解耦的能力表面。它们都服务于 Agent 的能力扩展，但进入 API 请求的位置不同。Skills 更像文档化的操作手册；Tools 更像可调函数的机器接口；MCP 则像一个动态工具提供者，具体暴露哪些函数取决于已经配置的 MCP servers。

以 Codex 的 Skills 为例，skill summary 会出现在 message content 中。讲者描述的场景是：用户发出第一条 query 后，远端模型根据 query 匹配合适的 Skill；若需要加载某个具体 Skill，系统会通过 **Function Call** 执行类似 cat 的 shell 命令，读取完整 SKILL.md；随后完整文件内容作为 FunctionOutput message 回到上下文。

Skills 的进入路径

Skill 的关键载体是 message content：先是摘要进入 user message 附近的上下文，随后完整 SKILL.md 可以通过 Tool 调用读入，并作为 FunctionOutput message 注入。这个过程体现了 progressive loading：先给模型目录级线索，再按需加载完整说明。

5.2 Tools/MCP 通过 API 的 tools 字段暴露

Tools 与 MCP 的路径不同。讲者明确说，MCP 或其他 built-in Tools 的定义放在 API 的 tools 字段里，以 **Tools Schema** 的方式进入请求。Tools Schema 告诉模型：有哪些工具可以调用，每个工具叫什么，需要什么参数，返回结果如何进入后续上下文。

这意味着，一个 Skill 文档可以告诉模型“遇到某类任务时应使用 web search、update plan 或 spawn agents”，但这句话本身不会创建对应的工具 schema。若 API 请求的 `tools` 字段没有注册这些工具，模型只能知道文档中的建议，无法真正发起对应 Function Call。

常见误解：Skill 提到 Tool 不等于注册 Tool

SKILL.md 可以描述何时使用某个 Tool，这属于 instruction。Tool 是否可调用，取决于 API `tools` 字段里是否存在对应 schema。把这两层混成一层，会误判 context 成本，也会误判 Agent 的真实行动边界。

5.3 为什么 Skills 不会自动降低 Tools 的 Context 成本

讲者特别强调：Skills 的出现不会天然简化 MCP 或 Tools 占用的 context，除非 runtime 明确做了相关设计。原因来自二者的正交位置。Skill summary 和完整 SKILL.md 主要进入 message list；Tools/MCP schema 主要进入 `tools` 字段。前者可以教模型“该如何想、何时做、按什么流程做”，后者提供“可调用接口的结构化定义”。

一个直观对照是：Skill 像一本“维修手册”，Tools Schema 像工具箱里每件工具的“可插拔接口说明”。手册可以写“需要拧螺丝时使用螺丝刀”，但工具箱里仍然要真的有螺丝刀，并且接口说明要让使用者知道它如何被拿起、如何被使用。MCP server 则更像外接工具柜：接入哪些柜子，决定 runtime 能展示哪些工具。

MCP 的动态性

MCP (Model Context Protocol) 在这里可理解为动态工具来源。它指向一种机制：runtime 从已配置 server 中发现和暴露函数。因此，同样叫“有 MCP 支持”的 Agent，在不同配置下可见的 Tools Schema 可能完全不同。

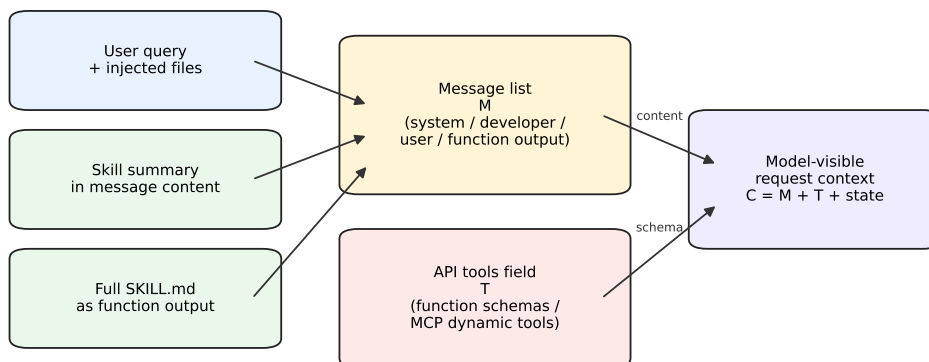
5.4 Skills-vs-Tools 图示

下面的生成图把本章最关键的结构压缩到一张图里。左侧是 message content 通道：`agents.md`、skill summary、完整 SKILL.md、FunctionOutput；右侧是 API `tools` 字段通道：built-in Tools、MCP functions、custom functions、Tools Schema。两条通道最终共同构成同一个 model-visible request context。

5.5 本章小结

本章把 Skills 与 Tools 的位置差异拆开：Skills 是文档化能力，主要通过 message content 与 FunctionOutput 进入上下文；Tools/MCP 是可调能力，主要通过 API `tools` 字段以 Tools Schema 进入上下文。二者都能扩展 Agent，但它们的载体、成本和故障模式不同。后续读 Codex 的请求结构时，必须持续区分“说明已经进入消息”和“工具已经注册可调用”这两件事。

Two context surfaces in a Codex-like agent



A Skill can describe when to use a tool, while the callable tool still needs an API schema.

图 2: Skills 与 Tools/MCP 的两个上下文入口: Skill 进入 message list, Tool schema 进入 API tools 字段。

6 Agent Loop: 从输入到动作再到持久化

6.1 Agent Runtime 是舞台, Agent Loop 是流程

讲者在这里把两个容易混淆的概念并排定义: **Agent Runtime** 是 Agent 执行的运行时环境; **Agent Loop** 是一条消息进入系统之后, 被转化为动作和回复的循环流程。若用剧场类比, runtime 是舞台、灯光、道具、后台和排练规则; loop 是演员每一次上场、接话、行动、谢幕并留下记录的过程。

在视频中的 **OpenClaw** 示例里, runtime 包括 workspace、启动时注入的 bootstrap 文件、built-in Tools、Skills、session transcript 的存储位置, 以及若干 markdown 文件, 例如 **AGENTS.md**、**SOUL.md**、**TOOLS.md**。这些材料构成 Agent 运行的环境和装备。loop 则从一条真实进入的消息开始, 经过 intake、context assembly、model inference、tool execution、streaming replies 和 persistence。

agent runtime 与 agent loop

- agent runtime: agent 执行的运行时环境(舞台)
 - 比如 workspace、注入的 bootstrap 文件、内建工具、skills、session transcript 存储位置, 都属于 runtime 的一部分。
 - markdown: [AGENTS.md](#) / [SOUL.md](#) / [TOOLS.md](#)
- agent loop
 - 当一条消息真的进来之后, 系统如何把它变成动作和回复。
 - intake → context assembly → model inference → tool execution → streaming replies → persistence.

图 3: **Agent Runtime** 与 **Agent Loop** 的区别: runtime 是执行舞台, loop 是消息进入后被组织成上下文、推理、工具执行、流式回复和持久化的流程。²

²视频画面时间区间: 00:06:10–00:06:30。若为裁剪图, 时间区间仍对应原视频画面。

Runtime 与 Loop 的最小区分

Agent Runtime 回答 “Agent 在什么环境里运行、有哪些材料和工具可被准备”。**Agent Loop** 回答 “一条消息进来之后，系统按什么顺序把它变成模型推理、工具调用、回复和本地记录”。

6.2 从 Intake 到 Persistence 的控制流

Agent Loop 可以压缩成一条工程流水线。为了避免把它误解成一个单步函数，下面用两行表示同一条流程：

$$L: \text{input} \rightarrow \text{queue} \rightarrow \text{context assembly} \rightarrow \text{model inference} \\ \rightarrow \text{function call} \rightarrow \text{tool execution} \rightarrow \text{streaming response} \rightarrow \text{persistence}$$

- *input*: 用户消息或上游系统事件进入 Agent。
- *queue*: 消息可能进入队列，等待 runtime 安排执行。
- *context assembly*: runtime 准备 prompt injection、启动文件、skill summary、memory、已有 transcript 和 tool schema。
- *model inference*: 模型基于当前可见上下文推理下一步。
- *function call*: 若需要外部动作，模型生成结构化工具调用。
- *tool execution*: runtime 执行对应 Tool，例如 shell、web search、apply patch、spawn agents。
- *streaming response*: 模型或 runtime 将结果以流式形式返回给用户界面。
- *persistence*: 会话、工具结果或 transcript 被保存到本地，供后续步骤或调试使用。

这个流程说明，Agent 的“会做事”来自一串接口配合。模型负责根据上下文选择下一步；runtime 负责把选择转成真实动作，再把动作结果变成后续上下文。若其中任一环没有正确暴露或持久化，Agent 就会出现“知道该做什么，却无法调用工具”或“做过动作，却无法在下一轮引用结果”的问题。

Prompt Injection 在这里的中性含义

本讲中的 **Prompt Injection** 指 runtime 主动注入的启动材料，例如项目规则、技能摘要、工具说明和 memory 片段。它是体系结构术语，并不必然等同于安全语境里的恶意 prompt injection。上下文工程需要同时关心有益注入与恶意注入，因为二者最终都会争夺模型可见上下文。

6.3 Function Call 把 Loop 接到真实世界

在 Agent Loop 中，**Function Call** 是模型推理和外部执行之间的桥。模型本身只产生 token；当它输出一个结构化工具调用时，runtime 才会把这个意图交给真实工具执行。工具执行之后，结果通常会回到 message list，成为后续推理的一部分。

这正好把上一章的 Skills 机制串起来：Codex 发现需要某个 Skill 后，可以通过 Function Call 执行 shell 命令读取 SKILL.md，再把完整文本作为 FunctionOutput 放回 context。对于 built-in

Tools 和 MCP functions, loop 的机制相似：模型提出调用，runtime 校验并执行，结果返回，再进入下一轮上下文。

不要把 Loop 理解成一次性问答

Agent Loop 的关键在“反复”。一次回复可能包含多轮模型推理、多次工具调用、多次 function output 注入和多次状态更新。若把它看成普通聊天问答，会低估 runtime 在排队、注入、执行、流式输出和持久化上的工程复杂度。

6.4 转向 Codex Tools Schema

这一节结尾自然过渡到 Codex 的 **Tools Schema**。前面已经说明：runtime 决定环境与上下文，loop 决定消息如何流动，Function Call 决定外部动作如何接入。下一步要观察的就是 Codex 到底把哪些函数暴露给模型，以及这些函数如何出现在 API 请求的 `tools` 字段中。

讲者列举的方向包括 `update plan`、`spawn agents`、`execute command` 等内置能力。它们是 Agent Loop 的执行节点，不能当成孤立按钮：模型看到 schema，选择调用；runtime 执行函数；函数结果回到上下文；随后模型继续推理。理解这条链，才能看懂为什么 Tools Schema 是现代 Agent 体系的核心证据。

6.5 本章小结

本章完成了从“上下文暴露”到“执行循环”的衔接。Agent Runtime 是准备环境、启动材料、工具定义和本地状态的舞台；Agent Loop 是消息进入后反复经历 `intake`、`context assembly`、`model inference`、`Function Call`、`tool execution`、`streaming response` 和 `persistence` 的流程。它们共同解释了现代 Agent 的真实形态：模型提供推理，runtime 提供可见上下文与执行边界，loop 把推理和行动组织成可持续的工作流。

7 工具枚举、MCP 动态加载与落盘观察

7.1 为什么要把 Tools Schema 单独拿出来看

前几章已经建立了一个关键前提：现代 AI Agent 的能力需要进入 `model-visible request context`，模型才有机会使用它。到这里，问题从“能力是否存在”推进到“能力怎样被声明”。在 Codex 这样的系统里，**Tools Schema** 就是工具能力的声明书：它告诉模型当前有哪些可调用能力、每个能力叫什么、属于哪一类、需要怎样的结构化参数。

这件事看似只是 API 细节，实际决定了 agent 的行动边界。若把 API 请求类比成寄给模型的包裹，消息内容是包裹里的说明书，`tools` 字段则像一组可操作设备的接口卡。模型可以阅读说明书，也可以按接口卡发起 `Function Call`；两者进入上下文的位置不同，成本、权限和风险也不同。

Tools Schema 是行动面的显式化

Tools Schema 把“宿主系统能做什么”转成模型可见的结构化接口。它既是能力清单，也是约束清单：模型能生成调用意图，真实执行仍由 Agent Runtime 按权限、沙箱和本地环境完成。

7.2 Codex 观察到的五类工具模式

视频在 Codex 示例中先展示了若干内置函数。三个代表性例子分别是：

- `update_plan`：更新任务计划。
- `spawn_agent`：启动子代理。
- `exec_command`：执行本地命令。

随后，讲者把目前观察到的 `Tools Schema` 类型概括为五类：

- `function`
- `local_shell`
- `image_generation`
- `web_search`
- `custom`

这些类别可以理解为运行时给工具贴上的“能力标签”。

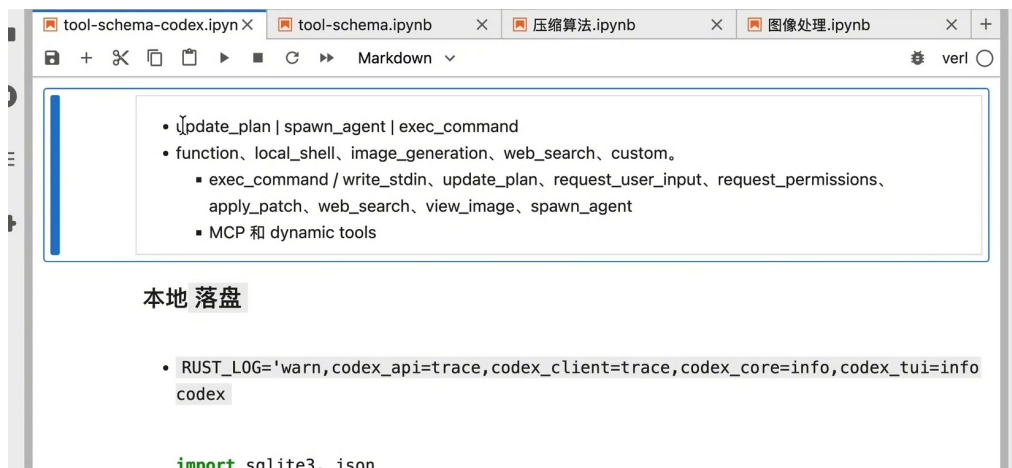


图 4：Codex 工具分类与本地落盘观察的开场画面。画面上方列出五类 `tool schema`，并把 `exec_command`、`update_plan` 等函数放入示例列表。³

这五类工具可以分成两层来理解。

- `function`：最常见的通用函数式工具。工具有名字、参数结构和返回值，适合 `update_plan`、`request_user_input`、`view_image` 等离散动作。
- `local_shell`：面向本地命令执行或命令会话交互。它把操作系统和工作区接入 `agent loop`，因此权限、沙箱和输出解析都很关键。
- `image_generation`：面向图片生成或编辑。它的输入往往是描述性提示词，输出是视觉资产，和普通文本函数的返回形态不同。

³视频画面时间区间：00:07:00–00:07:20。若为裁剪图，时间区间仍对应原视频画面。

- `web_search`: 面向网页或图片搜索。它把动态外部信息带回上下文，适合补充时效性事实。
- `custom`: 由运行时或应用层定义的特殊工具。它提醒读者：工具集合会随宿主系统扩展，不能按固定标准库理解。

五类工具的直观类比

可以把 `function` 看成标准办公按钮，把 `local_shell` 看成进入机器间的控制台，把 `web_search` 看成资料检索窗口，把 `image_generation` 看成视觉工作台，把 `custom` 看成项目专门安装的夹具。它们共同服务于同一个 agent loop，暴露方式和风险形态各不相同。

7.3 MCP 是动态工具来源

讲者紧接着强调，MCP 属于 dynamic tools。这里的 dynamic 指工具集合取决于当前配置了哪些 MCP server，以及这些 server 暴露了哪些 function。换句话说，MCP 像一个可插拔的工具供应层：同一个 Codex Runtime，在不同工作区、不同配置文件、不同账户权限下，模型可见的工具集合会发生变化。

用一个轻量公式可以压缩这一点。某一次请求中的工具集合 T 可近似写成：

$$T = T_{\text{default}} \cup T_{\text{mcp}}(\text{config}) \cup T_{\text{custom}}$$

T_{default} : 运行时默认提供的工具集合，例如命令执行、计划更新、补丁应用、图片查看等。

$T_{\text{mcp}}(\text{config})$: 由 MCP 配置决定的动态工具集合，配置不同，集合也随之变化。

T_{custom} : 项目、插件或宿主应用额外注入的自定义工具集合。

$\cup$: 集合并集，表示模型在该次请求中能看见的工具来自多个来源的合并。

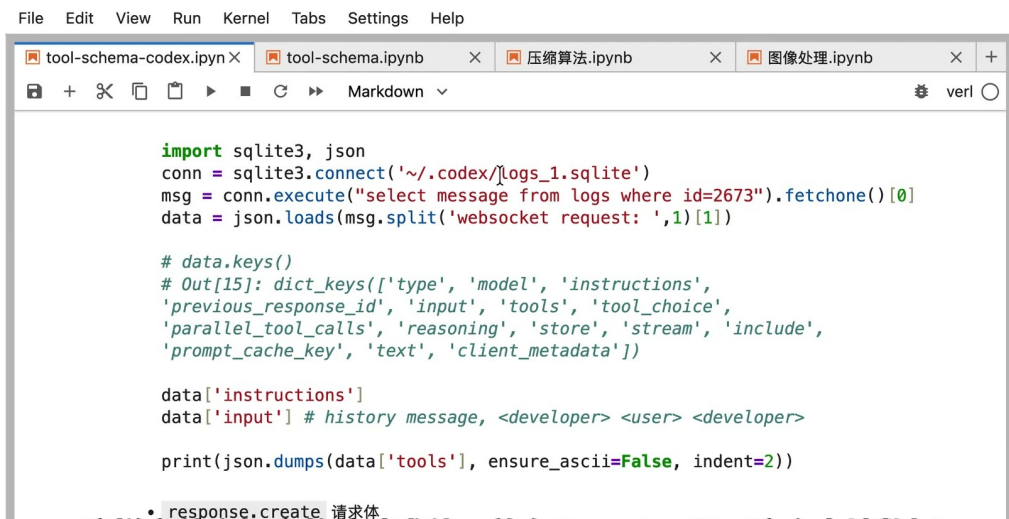
MCP 的动态性会影响复现

当一个调试记录说“模型可以调用某个工具”时，必须同时问清当时的 MCP server 配置、账户授权和运行时版本。只复制用户提示词，无法保证复制相同的工具面。

7.4 本地落盘：从黑箱请求到可检查证据

默认情况下，讲者观察到 API 层面的 Tools Schema 不会自动完整保存在本地。若需要检查一次真实 Codex 请求到底包含了哪些消息和工具，运行时需要打开相应的本地持久化设置。视频只可靠地说明“需要添加一个环境变量”，这里不写出具体的变量名，避免把画面里的实验设置误当成稳定公开接口。

打开持久化后，完整 request body 会进入本地 SQLite 数据库 `logs_1.sqlite`。讲者举了一个测试例子：启动后发送 `hello`，对应记录落在 id 2673。随后用 Python 读取该记录，把 websocket request 中的 JSON request body 解析出来，就能看到 `instructions`、`input`、`tools` 等字段。



```
import sqlite3, json
conn = sqlite3.connect('~/.codex/logs_1.sqlite')
msg = conn.execute("select message from logs where id=2673").fetchone()[0]
data = json.loads(msg.split('websocket request: ',1)[1])

# data.keys()
# Out[15]: dict_keys(['type', 'model', 'instructions',
# 'previous_response_id', 'input', 'tools', 'tool_choice',
# 'parallel_tool_calls', 'reasoning', 'store', 'stream', 'include',
# 'prompt_cache_key', 'text', 'client_metadata'])

data['instructions']
data['input'] # history message, <developer> <user> <developer>

print(json.dumps(data['tools'], ensure_ascii=False, indent=2))
```

图 5: 本地日志观察: 通过 SQLite 记录读取完整 request body, 并打印出请求中的关键字段。⁴

落盘日志与“模型记忆”分属两层

logs_1.sqlite 中的 request body 是本地调试和审计证据, 属于运行时保存的数据。模型当前能使用什么, 仍取决于本次 API 请求里实际携带的消息、工具 schema 和上下文链接。把本地日志、长期 memory、当前上下文混成一类, 会直接误判 agent 的能力边界。

7.5 本章小结

本章把 Codex 的工具面拆成三个观察点。第一, Tools Schema 是 API 层面的行动声明, 它和 Skill 文档进入上下文的位置不同。第二, 目前可见的工具类型包括 function、local_shell、image_generation、web_search、custom, 而 MCP 会根据 server 配置动态扩展工具集合。第三, 本地落盘让 request body 从推测对象变成可检查证据, 尤其适合验证 instructions、input 和 tools 字段究竟怎样出现。

对工程实践而言, 最硬的结论是: 讨论现代 agent 的能力时, 必须同时看“文档说明进入了哪里”和“工具 schema 注册到了哪里”。只看聊天文本会漏掉行动接口, 只看工具列表又会漏掉指导模型如何使用工具的上下文。

8 Response API 请求体结构

8.1 从日志里看到 response.create

上一章讨论本地落盘的目的, 是把一次 Codex 请求从抽象描述变成可检查的 request body。视频继续把这个 request body 定位为 response.create 请求。对读者来说, 最重要的变化是视角转换: 现代 agent 的一次回答远远超出孤立文本生成, 它是一种带有系统指令、消息列表、历史链接和工具声明的结构化 API 调用。

讲者强调, instructions 是独立的顶层指令, 基本相当于 system prompt。input 是完整的 Message List, 里面可以包含 developer、user、assistant, 以及 function call output。previous response

⁴ 视频画面时间区间: 00:08:00-00:08:20。若为裁剪图, 时间区间仍对应原视频画面。

id 用于上下文链接。tools 则承载工具 schema。这个结构解释了为什么前几章反复区分 message content 和 API tools 字段：二者在同一个 request body 内并列存在，各自承担不同任务。

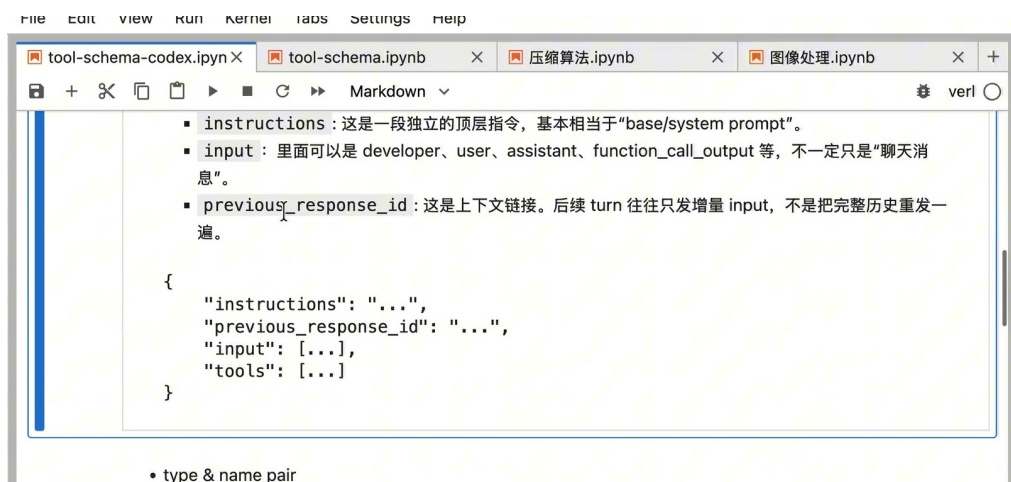


图 6: response.create 请求体的教学骨架：顶层 instructions、previous_response_id、input 消息列表与 tools 字段共同构成一次模型请求。⁵

8.2 四个主轴：instructions、input、previous response、tools

可以把 response.create 的主线拆成四根柱子。

1. instructions：顶层系统性约束，负责框定模型行为边界、输出风格、任务原则和安全约束。
2. input：消息列表，承载 developer/user/assistant 等角色消息，也可以承载工具执行后的 function output。前文提到的 Skill summary、完整 SKILL.md 内容、agents.md 之类材料，往往就通过消息内容进入这里。
3. previous_response_id：Response API 的上下文链接机制。后续 turn 可以挂载之前的 response id，使运行时把连续交互连接成链。
4. tools：工具 schema 列表，告诉模型当前有哪些工具可被调用，以及调用时需要怎样的结构化参数。

消息列表像会议记录，tools 字段像设备清单

input 主要告诉模型“到目前为止谈了什么、遵守哪些角色指令、工具返回了什么观察”。tools 主要告诉模型“现场有哪些设备可以申请调用”。会议记录再详细，也不会自动生成设备接口；设备清单再完整，也无法替代如何使用设备的上下文说明。

为了帮助读者形成结构感，可以用下面的概念骨架表示。它只保留本讲需要的字段，实际日志中还可能包含 model、stream、reasoning、tool_choice 等控制项。

```
1 {  
2   "instructions": "...",  
3   "previous_response_id": "...",  
4   "input": [  

```

⁵视频画面时间区间：00:09:00–00:09:20。若为裁剪图，时间区间仍对应原视频画面。

```

5     {"role": "developer", "content": "..."},
6     {"role": "user", "content": "..."},
7     {"role": "assistant", "content": "..."},
8     {"type": "function_call_output", "call_id": "...", "output": "..."}
9 ],
10 "tools": [
11     {"type": "function", "name": "exec_command", "parameters": {...}},
12     {"type": "web_search", "name": "web_search"}
13 ]
14 }

```

Listing 1: response.create 请求体的概念骨架

8.3 一个简化公式：Context 由消息、工具和运行时链接组成

为避免把上下文工程误解为“写一段更长的提示词”，这里沿用前文的教学用简化公式。模型在某次调用中实际可见、可用的上下文 C_{visible} 可以近似写作：

$$C_{\text{visible}} \approx M \cup T \cup R$$

C_{visible} ：本次调用中模型可见、可用的整体上下文，包含文本材料、工具 schema 和运行时提供的链接信息。

M ：Message List，即 input 中的系统性说明、developer/user/assistant 消息、function output 等。

T ：工具 schema 集合，即 tools 字段中注册的可调用能力。

R ：运行时管理的上下文链接和状态，例如 previous_response_id、队列、日志、workspace 状态的相关引用。

$\cup$ ：集合合并。这里是概念化写法，用于说明上下文来自多个表面，具体 API 实现并不等同于数学集合运算。

Context Engineering 的结构化定义

在本讲语境里，Context Engineering 指设计 M 、 T 、 R 三类材料怎样进入请求、何时进入请求、以什么粒度进入请求。它覆盖 prompt 编写，也覆盖工具注册、Skill 渐进加载、历史链接和本地持久化观察。

8.4 容易误读的地方

第一，instructions 与 input 的分工需要分清。前者更像顶层行为框架，后者更像本轮及历史交互材料。若把所有内容都塞进用户消息，运行时就失去了分层表达约束和材料的机会。

第二，previous_response_id 让后续 turn 可以通过链接复用历史关系。它降低了每一轮重新发送完整历史的需求，可是它也带来调试难度：只看当前 input 片段，可能看不全整条上下文链。

第三，tools 字段里的 schema 只是“允许模型提出调用”的声明。工具是否真正执行、执行到哪一步、是否需要用户授权、是否被沙箱拦截，仍由 Agent Runtime 和宿主环境决定。

不要把 request body 看成普通聊天记录

普通聊天记录只解释“模型看见了哪些文本”。`response.create request body` 还解释“模型被允许调用哪些工具、如何承接上一轮 `response`、工具输出怎样回流为新消息”。漏掉这些结构，就无法解释现代 coding agent 的真实行为。

8.5 本章小结

本章把 `response.create` 请求体拆成可教学的四个主轴：`instructions` 给出顶层指令，`input` 承载完整消息列表，`previous_response_id` 建立上下文链接，`tools` 暴露可调用工具 schema。这个结构把 Skills 与 Tools 的正交关系落到了具体字段上：Skill 内容主要进入消息列表，Tool/MCP 能力主要进入 `tools` 字段。

因此，分析 Codex 这类 agent 时，单看最终回答远远不够。更可靠的做法是检查请求体：有哪些角色消息，哪些 function output 已经回流，哪些工具 schema 被注册，上一轮 `response` 怎样链接。只有把这四类证据合起来，才能判断模型为什么能做某个动作，以及为什么有些看似存在的能力在当轮请求里仍然不可用。

9 Codex 默认工具与多 Agent 能力

9.1 没有配置 MCP 时的默认函数清单

视频最后一段把前面的结构讨论落到具体工具清单上。讲者说明，在没有配置任何 MCP server 的情况下，观察到 Codex 默认已有约 13 个 function。这一点很有教学价值：即使动态 MCP 工具为空，Agent Runtime 也会提供一组基础能力，让 coding agent 可以执行命令、维护计划、请求用户输入、编辑文件、浏览网页、查看图片，并启动子代理。

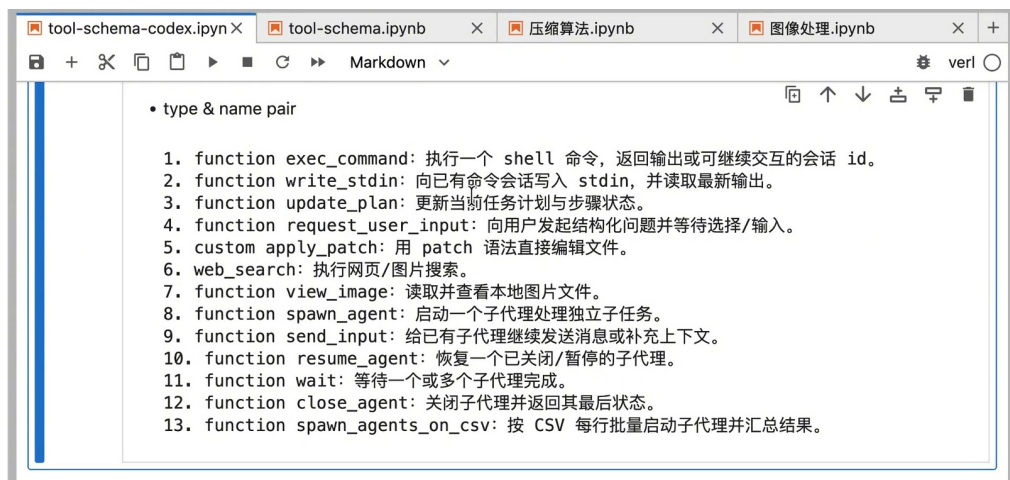


图 7: 默认函数列表上半部分：`exec_command`、`write_stdin`、`update_plan`、`request_user_input`、`apply_patch`、`web_search`、`view_image` 等基础工具构成单代理执行面的核心。⁶

这些工具可以按 workflow 角色重组。

⁶ 视频画面时间区间：00:09:30-00:09:50。若为裁剪图，时间区间仍对应原视频画面。

- 执行与观察: `exec_command` 执行 shell 命令, `write_stdin` 向已有命令会话写入标准输入并读取最新输出。
- 计划与协商: `update_plan` 更新当前任务计划和步骤状态, `request_user_input` 发起结构化问题并等待用户选择或输入。
- 编辑与检索: `apply_patch` 用 patch 语法直接编辑文件, `web_search` 执行网页或图片搜索, `view_image` 读取并查看本地图片文件。

默认工具让 agent 从“会说”进入“会做”

如果没有命令执行、补丁编辑、图片查看、计划更新这些基础工具, coding agent 只能给出建议。默认函数把建议转成可执行动作, 把外部观察再带回上下文, agent loop 才能连续推进。

9.2 单代理工具：执行、编辑、检索、协商

`exec_command` 与 `write_stdin` 负责把本地环境接入 agent loop。前者启动命令, 后者让长时间运行或交互式命令继续通信。它们使模型能够从“根据记忆推测”转向“基于终端输出判断”。这类工具的返回值也会成为后续 Message List 的一部分, 影响下一步推理。

`update_plan` 看似只是界面辅助, 实际上是长任务控制工具。现代 coding agent 往往需要跨越阅读、修改、测试、复查多个阶段。计划工具把这些阶段显式化, 让用户和 agent 都能看到当前任务处于哪一步、哪些步骤已经完成、哪些步骤仍在等待。

`request_user_input` 负责在关键歧义处让用户介入。它和普通追问的差异在于结构化: 问题、选项、推荐路径、自动处理策略都能进入运行时控制流。对高风险操作或需求不完整的任务, 这个工具比模型单方面猜测更稳。

`apply_patch` 是文件编辑工具。它把修改约束成明确的 patch, 避免停留在模型口述“请改这里”的层面。`web_search` 与 `view_image` 则分别面向动态网页事实和视觉证据。本项目生成 PDF 时要求直接检查图片, 这正体现了 `view_image` 的价值: 图像文件名只能辅助定位, 图像内容本身才是证据。

工具越接近真实环境, 越需要权限边界

`exec_command`、`apply_patch`、`web_search` 都会把模型意图转成外部动作。它们带来效率, 也扩大误操作面。可靠的 runtime 需要沙箱、审批、日志和用户确认, 把“能调用”与“允许执行”分开治理。

9.3 多 Agent 工具：把长任务拆成可汇总的并行工作

默认函数列表的下半部分转向多 Agent。讲者列出的函数覆盖五类生命周期动作: 启动子代理、继续补充上下文、恢复子代理、等待完成、关闭子代理。画面还提到最近新增的 CSV 批量启动能力。这些函数说明 Codex 的工具面已经超出单个模型回合, 它可以管理多个子代理的生命周期。

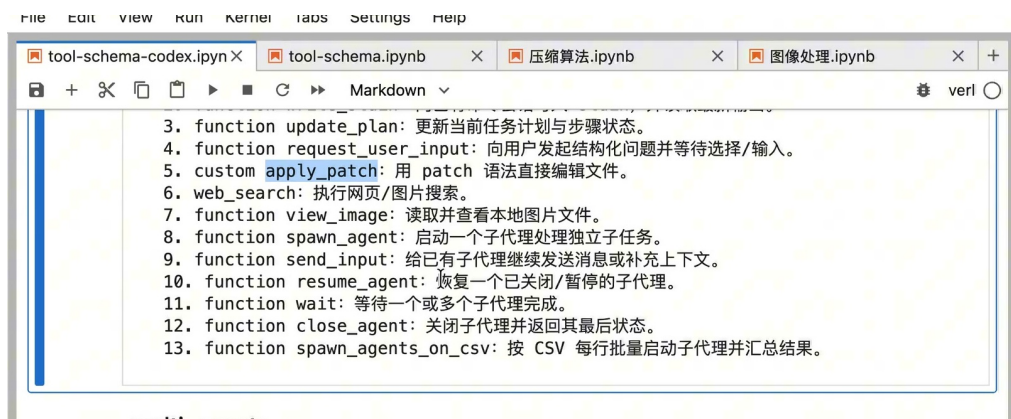


图 8: 默认函数列表下半部分: 多 Agent 相关函数覆盖启动、补充上下文、恢复、等待、关闭, 以及按 CSV 批量启动并汇总结果。⁷

`spawn_agent` 用于启动一个子代理处理独立子任务。它适合上下文相对独立、产出可以合并的工作, 例如一组章节并行写作、一组文件独立审查、一组候选方案分别验证。

`send_input` 负责向已有子代理继续发送消息或补充上下文, 避免任务中途发现信息缺口时只能重启。`resume_agent` 用于恢复已关闭或暂停的子代理, 适合长任务在会话切换后继续推进。

`wait` 与 `close_agent` 管理收束阶段。前者等待一个或多个子代理完成, 后者关闭子代理并返回最终状态。若把主代理看作项目负责人, 子代理就是多个专业执行者; 主代理的关键职责是派发边界清楚的任务、等待产出、合并冲突、做最终一致性判断。

`spawn_agents_on_csv` 则把多 Agent 调度进一步批处理化。CSV 的每一行可以对应一个子任务, 运行时批量启动子代理并汇总结果。这对视频转 PDF、批量评测、并行审查等任务很有吸引力, 因为任务天然可以分片, 同时又需要最后统一整合。

多 Agent 的价值在于隔离与汇总

多 Agent 的价值在隔离上下文、并行推进和引入独立检查。它也会带来合并成本: 术语可能不一致, 章节边界可能重叠, 局部结论可能互相冲突。因此主代理或一致性代理必须承担集成责任。

9.4 从工具清单回到 Context Engineering

默认函数清单把本讲的核心命题压得很清楚: 现代 coding agent 的能力来自模型、上下文和运行时三者的耦合。模型生成调用意图, Tools Schema 让调用意图有结构, Agent Runtime 执行动作并把结果回流为新上下文。多 Agent 工具进一步说明, runtime 不只执行单步命令, 还能调度多个 agent loop。

这也解释了为什么本项目要求多个子代理分工写作。一个长视频的 PDF 包含取证、抽帧、写作、校对、编译、布局检查等多个阶段, 单次摘要无法覆盖这些环节。把它们拆给独立子代理, 本质上就是把 `spawn_agent` 这一类运行时能力转化成生产流程。

9.5 本章小结

本章梳理了 Codex 在无 MCP server 配置时观察到的默认函数。单代理工具覆盖命令执行、会话输入、计划管理、用户澄清、补丁编辑、网页搜索和图片查看; 多 Agent 工具覆盖启动、补充上

⁷ 视频画面时间区间: 00:10:00-00:10:20。若为裁剪图, 时间区间仍对应原视频画面。

下文、恢复、等待、关闭和 CSV 批量启动。它们共同说明：`tools` 字段属于 agent runtime 的行动接口，具有直接工程含义。

对读者最重要的 takeaway 是，工具清单既揭示能力，也揭示治理问题。工具越强，越需要边界；子代理越多，越需要一致性检查；本地落盘越完整，越需要注意敏感信息。Context Engineering 的成熟形态，就是把这些能力、边界和证据组织成可审计的运行时流程。

10 总结与延伸

10.1 全片核心：能力必须进入模型可见请求

本讲围绕一个朴素却硬核的事实展开：现代 AI Agent 的能力并不只来自 foundation model。模型之外还有 Agent Runtime，它负责准备 `instructions`、`input` 消息列表、`tools` 字段、Skill 摘要、完整 `SKILL.md`、MCP server 暴露的函数、历史 `response` 链接，以及本地持久化证据。模型只能基于本次请求中实际暴露的内容推理和行动。

把全片压缩成一句话：Context Engineering 是 runtime 管理模型可见世界的工程。它问的核心问题包括：哪些文档进入消息内容，哪些工具进入 `tools` 字段，哪些历史通过 `response id` 连接，哪些观察作为 `function output` 回流，哪些运行时数据需要落盘供审计。

本讲最终命题

现代 agent 的可用能力 = 模型能力 + runtime 注入的消息材料 + API 层注册的工具 schema + loop 中回流的观察结果。缺少任一环，系统表现都会发生实质变化。

10.2 Skills、Tools、MCP 的统一视角

前半部分讨论 Skills 与 Tools 的位置差异，后半部分用 Codex 的 request body 验证这一点。Skill 是文档型能力：先以摘要形式进入消息内容，需要时再把完整 `SKILL.md` 通过工具调用读入，作为 `function output` 回到消息列表。Tool 是 API 型能力：它以 schema 的形式进入 `tools` 字段，让模型知道可以发起哪些结构化调用。MCP 则是动态工具来源：它根据配置的 MCP server 把更多函数暴露到工具面。

因此，三者处在不同层级，含义也不同。更准确的统一视角是：它们都服务于同一个 model-visible request context，只是进入路径不同。

- Skills 主要回答“模型应该怎样做这类任务”，入口偏 `message content`。
- Tools 主要回答“模型现在可以申请调用哪些动作”，入口偏 `API tools field`。
- MCP 主要回答“外部系统怎样动态提供工具”，入口取决于 MCP server 配置和 runtime 注册过程。

最容易犯的架构误判

看到一个 Skill 文档里提到某个工具，并不意味着该工具 schema 已经注册进 `tools` 字段。看到某个 MCP server 已经安装，也不意味着本轮请求已经暴露了它的所有函数。真正的证据是 request body。

10.3 用一个 Agent Loop 公式串起来

现代 agent 的执行过程可以写成一个简化 loop:

$$L : x \rightarrow (M, T, R) \rightarrow (y, a) \rightarrow o \rightarrow M'$$

L : Agent Loop, 即从用户输入到响应、动作、观察回流的重复控制流。

x : 用户输入或外部任务事件。

M : 消息列表, 包含 system/developer/user/assistant/function output 等材料。

T : 工具 schema 集合, 包含默认函数、MCP 动态工具和自定义工具。

R : 运行时状态与链接, 例如 `previous_response_id`、队列、日志和 workspace 相关状态。

y : 模型生成的自然语言响应。

a : 模型提出的结构化工具调用意图。

o : 工具执行后的观察结果, 例如命令输出、网页检索结果、图片检查结论或子代理状态。

M' : 观察结果回流后的新消息列表, 为下一轮 loop 提供材料。

这个公式的价值在于提醒读者: agent 的“智能”不只发生在模型内部。很多关键质量来自 M 、 T 、 R 的设计。文档注入太多会挤压上下文, 工具 schema 太多会造成选择混乱, 历史链接不清会让调试困难, 观察结果不落盘会让复现无从谈起。

10.4 工程启示: 从聊天界面到可编程运行时

本讲最有价值的地方, 是把 Codex 这类产品从“聊天机器人”重新看成“可编程运行时”。`update_plan` 让长任务具备显式状态; `apply_patch` 让代码修改具备可审查补丁; `web_search` 与 `view_image` 让外部事实和视觉证据进入上下文; `spawn_agent`、`wait`、`close_agent` 让任务可以并行分解再统一汇总。

这也解释了为什么单纯追求“更长上下文”并不能解决全部问题。长上下文只是容量, Context Engineering 关注的是选择、结构、时机和证据。真正的生产系统要回答更细的问题: 哪些 Skill summary 先给模型看, 什么时候读完整 Skill, 哪些 MCP 工具在当前任务里有必要暴露, 哪些日志应该保留供复盘, 哪些高风险工具需要用户审批。

一个实用判断标准

当一个 agent 行为无法解释时, 优先查四件事: 本轮 `instructions` 写了什么, `input` 里有哪些消息和 function output, `tools` 字段注册了哪些 schema, 运行时有没有通过日志或 `response id` 连接隐藏的历史。很多看似“模型突然变聪明或变笨”的现象, 都会在这四类证据里找到结构性原因。

10.5 继续追问: 版本、权限、日志和安全

本讲使用的是一次具体观察, 工具列表、request body 字段和本地持久化行为都可能随 Codex 版本、运行时配置和账户权限变化。对工程团队来说, 这一点不应被轻描淡写。默认工具可能新增或改名, MCP server 可能暴露不同函数, 本地日志可能包含敏感请求体, 工具执行可能触碰文件系统、网络或外部服务。

因此, 后续实践可以沿四条线继续深入。

1. 版本线: 记录 Codex Runtime、CLI、API 和 MCP server 的版本, 避免把一次观察误写成永久契约。

2. 权限线：区分“模型看见工具 schema”和“运行时批准真实执行”，把审批、沙箱和审计放进流程。
3. 上下文线：评估 Skill、memory、function output、历史链接对上下文窗口的占用，避免把所有材料一次性塞满。
4. 证据线：对关键任务保留 request body、工具输出、子代理结果和最终补丁，让复盘能够回到事实。

视频最后真正值得保留的教学收束，是默认函数清单和多 Agent 能力本身：它们展示了现代 coding agent 如何从单轮文本生成扩展为可执行、可编排、可审计的 runtime 系统。例行感谢和私有仓库推广不进入教学主体，因为它们无法增加对 Context Engineering、Tools Schema 或 Agent Loop 的理解。

读者把这套笔记用于实际工程时，可以把最后四条追问当作验收清单：先确认版本，再确认权限；先看上下文窗口，再看工具 schema；先保存证据，再解释行为。这样复盘 agent 行为时，分析会落到 request body、工具输出和运行时状态这些硬证据上，避免停留在“模型突然变聪明”或“模型突然变笨”的模糊描述里。

10.6 复盘操作清单

把本讲转成团队实践时，可以按下面五步检查一次 agent 行为。这个清单适合用于故障复盘、能力边界说明，也适合用于设计新的 Skill 或 MCP server。

1. 看入口：确认用户 query、developer 约束、项目规则和 Skill summary 是否进入了当前 message list。
2. 看工具：确认 tools 字段中有哪些 schema，哪些来自默认函数，哪些来自 MCP 或 custom provider。
3. 看执行：确认每次 function call 的参数、审批状态、运行结果和 function output 是否形成可追踪链条。
4. 看状态：确认 previous response id、队列、workspace、日志和 memory 摘要有没有影响当前判断。
5. 看证据：保留关键 request body、终端输出、网页或图片证据、子代理结果和最终补丁，方便后续审计。

只要这五步能回答清楚，Context Engineering 就从抽象口号变成了可检查的工程过程；如果其中任何一步缺证据，agent 的行为解释就会出现硬缺口。